Fieldwork

Overview

Fieldwork evaluates whether coding agents can infer unstated requirements while completing real engineering work for large enterprises in private production codebases. The dataset includes 1,000+ tasks spanning full-stack application development, data engineering, SRE and infrastructure, production debugging, and performance optimization. Each environment is a production-ready codebase that lets agents run the application, spin up databases and microservices, use browsers, and test changes end-to-end as a senior engineer would.

Task Creation

Tasks originate from real customer requests and the coding agent sessions used to address them (all using Astra, Fable, or Opus). These sessions required human intervention when agents missed requirements they were expected to infer from the request or codebase: engineers corrected agents, or customers exposed missed requirements. Those corrections shape the task instructions and the verifiers.

Our pipeline brings together the original request, Slack conversations, Granola notes, engineering discussions, PRs, and subsequent feedback to reconstruct a coherent task. It preserves customer context and the engineer's direction across the work, rather than treating each PR as a standalone problem.

Some instructions are concise, leaving the agent to infer unstated requirements from the codebase:

```
Make the Equipment Type column editable in the equipment grid without compromising allocation
consistency.
```

Others include the original request and surrounding engineering discussion, leaving the agent to translate that context into a complete implementation:

```
# Stop phases from being allocated more equipment than they demand

A planner found a phase with 28 generators allocated against a demand of 14,
and nothing in the app stops that. App Allocation on the Equipment page
checks that the phase needs that equipment type at all...

From the thread where we settled what to do:

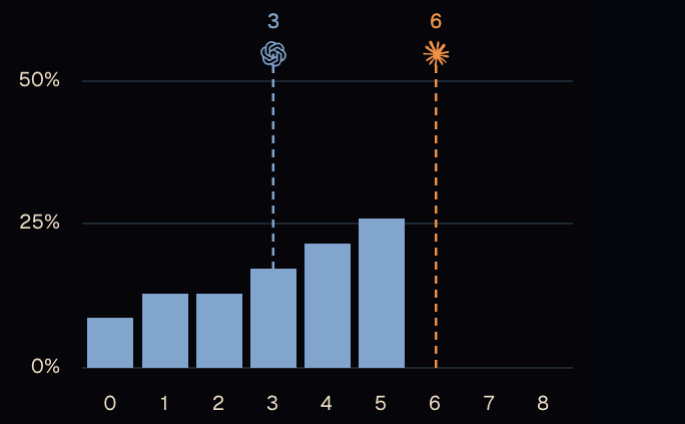
> I think it makes sense to just block the allocation action if it leads to
> an overallocation … Ya I think we just prevent either action with a note
> explaining over allocation state is not allowed
>
> Don't think it would make sense for the app to choose randomly which
> specific supply to unallocate. I guess u just unallocate some manually and
> then reduce demand

Make this a rule the app enforces: for each equipment type, what is
allocated to a phase must not exceed what the phase's lineups demand. It has
to hold for every way the app changes what is allocated or what...
```

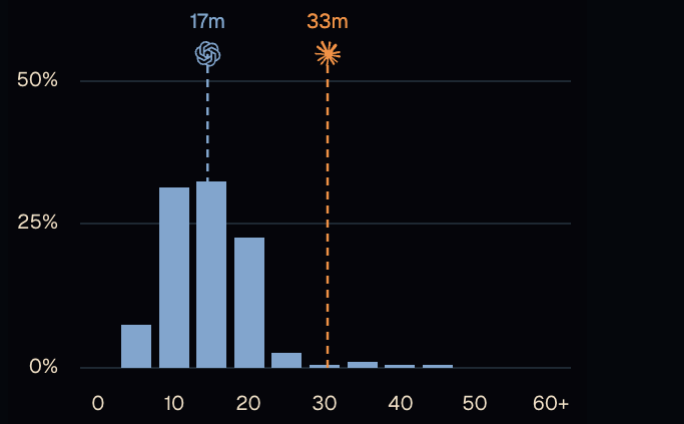
Results

Model	Pass rate	Median runtime
GPT 6 Astra xhigh	38.6%	16m 43s
Claude Opus 5.5 xhigh	65.8%	32m 49s

Trials passed per task



Trial runtime



Trials pass when every hidden verifier check passes (binary reward). Results cover a sample of tasks where Astra passes one to five of eight trials, plus tasks Astra never passes that Opus solves at least once.

Engineering work

Category	Share
End-to-end feature development	65%
System design and architecture	8%
Data engineering and integrations	28%
Infrastructure and platform engineering	11%
Production debugging and reliability	34%
Refactoring and migration	11%
Performance engineering	7%

Systems exercised

Category	Share
Browser and user interface	63%
Application services and APIs	62%
Databases and persistent state	67%
Background jobs and orchestration	31%
External services and integrations	30%
Infrastructure and deployment	9%
Files and artifacts	27%

Sample Tasks

TASK	TRIALS	PASSED	RUNTIME	PASSED	RUNTIME
app-allocation-approval	PASS 19m · PASS 24m · PASS 24m · PASS 25m · PASS 27m · FAIL 20m · FAIL 22m · FAIL 22m	5 / 8	23m 10s	**6 / 8**	37m 44s
change-equipment-types	PASS 12m · PASS 14m · PASS 14m · PASS 15m · FAIL 12m · FAIL 12m · FAIL 13m · FAIL 14m	4 / 8	13m 27s	**8 / 8**	20m 20s
block-phase-overallocation	PASS 19m · PASS 19m · PASS 21m · FAIL 18m · FAIL 20m · FAIL 20m · FAIL 21m · FAIL 24m	3 / 8	20m 2s	**7 / 8**	44m 56s
rippling-roster-email-ingest	PASS 16m · FAIL 13m · FAIL 14m · FAIL 14m · FAIL 14m · FAIL 15m · FAIL 16m · FAIL 18m	1 / 8	14m 31s	**3 / 8**	23m 31s
migration-baseline-collapse	PASS 10m · PASS 11m · PASS 11m · PASS 12m · PASS 12m · FAIL 11m · FAIL 12m · FAIL 16m	**5 / 8**	11m 30s	2 / 8	34m 12s
version-diff-paging-site-identity	PASS 10m · PASS 13m · FAIL 12m · FAIL 12m · FAIL 12m · FAIL 13m · FAIL 13m · FAIL 13m	**2 / 8**	12m 20s	0 / 8	20m 10s
n1x-posthog-provisioning	PASS 8m · PASS 10m · FAIL 8m · FAIL 8m · FAIL 8m · FAIL 9m · FAIL 9m · FAIL 11m	2 / 8	8m 27s	**7 / 8**	19m 33s
n1x-pagerduty-slack-connection-drift	PASS 18m · PASS 19m · PASS 21m · PASS 21m · FAIL 15m · FAIL 20m · FAIL 22m · FAIL 37m	4 / 8	20m 15s	**8 / 8**	27m 58s
po-report-restore-saved-value	FAIL 13m · FAIL 13m · FAIL 13m · FAIL 15m · FAIL 16m · FAIL 17m · FAIL 18m · FAIL 19m	0 / 8	15m 27s	**7 / 8**	41m 34s
geo-responses-csv-export	FAIL 17m · FAIL 18m · FAIL 20m · FAIL 20m · FAIL 20m · FAIL 21m · FAIL 21m · FAIL 22m	0 / 8	19m 58s	**6 / 8**	43m 7s